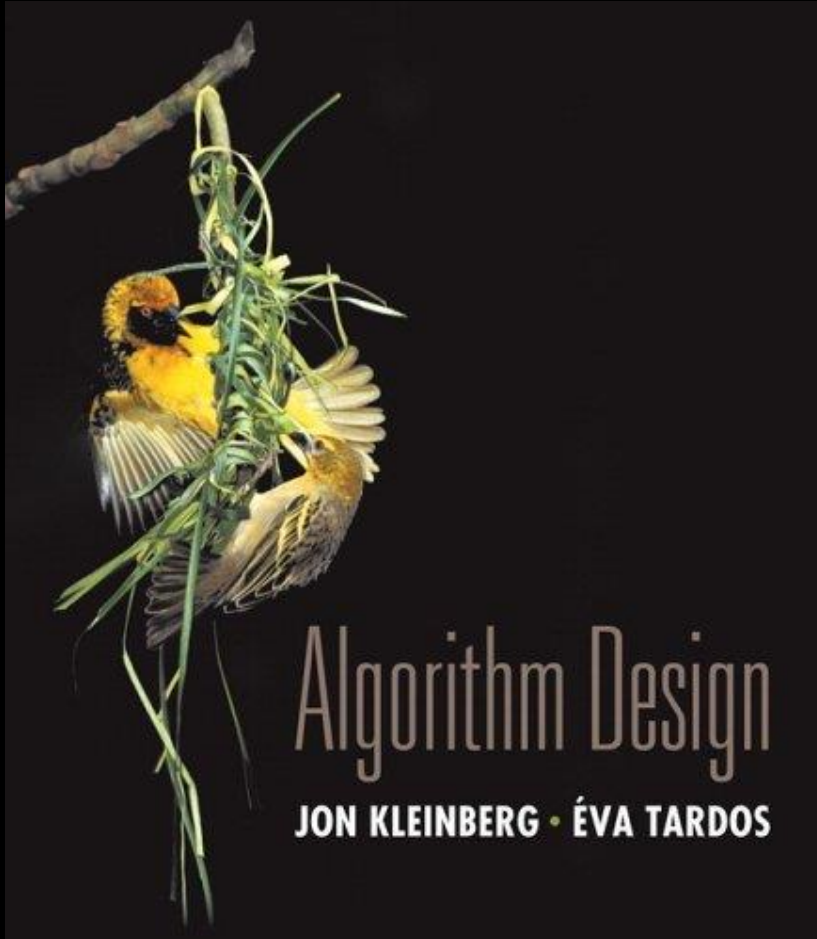


Chapter 5

Divide and Conquer



Slides by Kevin Wayne.
Copyright © 2005 Pearson-Addison Wesley.
All rights reserved.

Divide-and-Conquer

Divide-and-conquer.

- Break up problem into several parts.
- Solve each part recursively.
- Combine solutions to sub-problems into overall solution.

Most common usage.

- Break up problem of size n into **two** equal parts of size $\frac{1}{2}n$ in **linear time**.
- Solve two parts recursively.
- Combine two solutions into overall solution in **linear time**.

Consequence.

- Divide-and-conquer: $\Theta(n \log n)$

5.1 Mergesort

Mergesort

Mergesort.

- Divide array into two halves.
- Recursively sort each half.
- Merge two halves to make sorted whole.



Jon von Neumann (1945)

A	L	G	O	R	I	T	H	M	S
---	---	---	---	---	---	---	---	---	---

A	L	G	O	R
---	---	---	---	---

I	T	H	M	S
---	---	---	---	---

divide

A	G	L	O	R
---	---	---	---	---

H	I	M	S	T
---	---	---	---	---

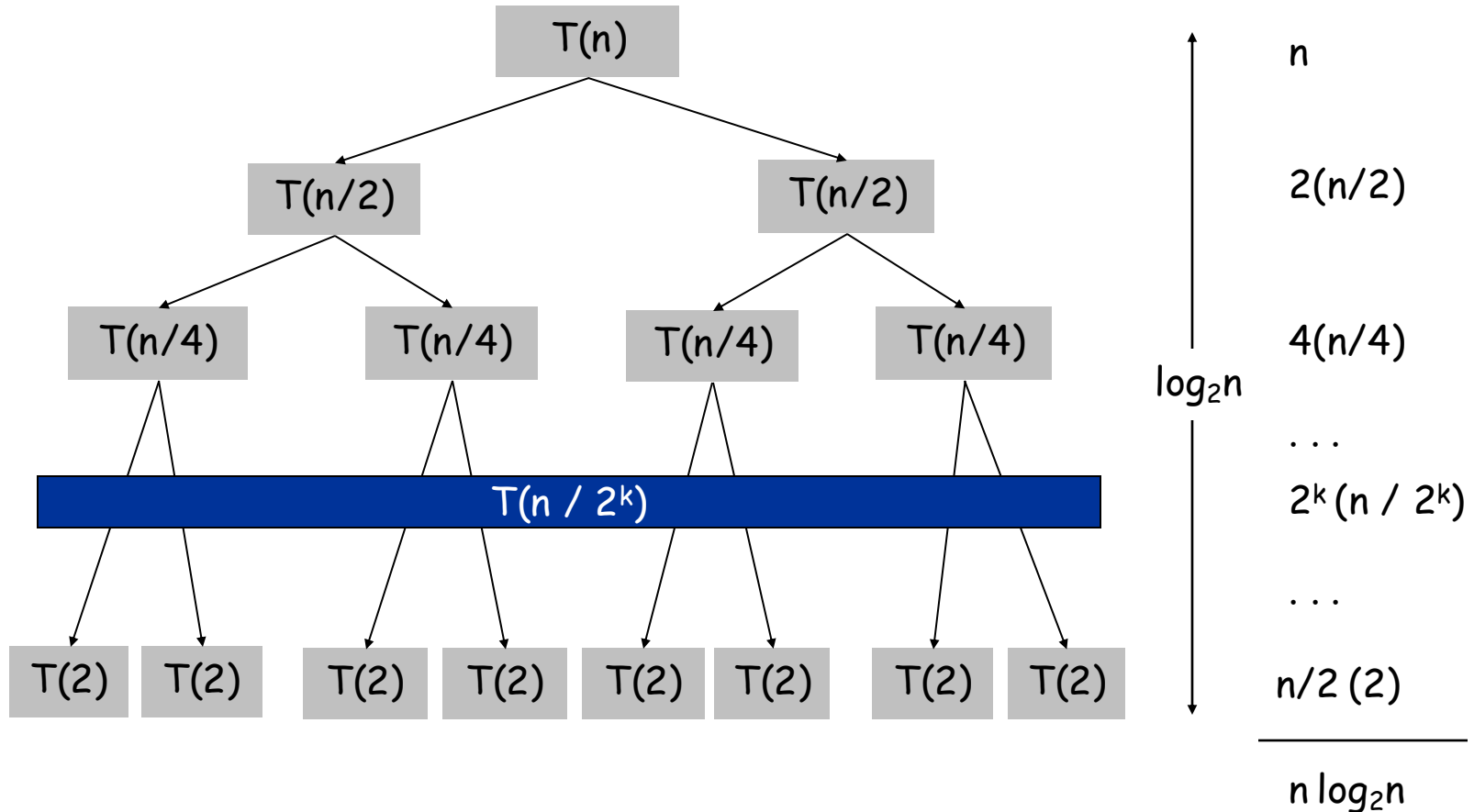
sort

A	G	H	I	L	M	O	R	S	T
---	---	---	---	---	---	---	---	---	---

merge

Proof by Recursion Tree

$$T(n) = \begin{cases} 0 & \text{if } n=1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$



Proof by Induction

Claim. If $T(n)$ satisfies this recurrence, then $T(n) = n \log_2 n$.

↑
assumes n is a power of 2

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

Pf. (by induction on n)

- Base case: $n = 1$.
- Inductive hypothesis: $T(n) = n \log_2 n$.
- Goal: show that $T(2n) = 2n \log_2 (2n)$.

$$\begin{aligned} T(2n) &= 2T(n) + 2n \\ &= 2n \log_2 n + 2n \\ &= 2n(\log_2(2n) - 1) + 2n \\ &= 2n \log_2(2n) \end{aligned}$$

Analysis of Mergesort Recurrence

Claim. If $T(n)$ satisfies the following recurrence, then $T(n) \leq n \lceil \log_2 n \rceil$.

$$T(n) \leq \begin{cases} 0 & \text{if } n = 1 \\ \underbrace{T(\lceil n/2 \rceil)}_{\text{solve left half}} + \underbrace{T(\lfloor n/2 \rfloor)}_{\text{solve right half}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

Pf. (by induction on n)

- Base case: $n = 1$.
- Define $n_1 = \lfloor n / 2 \rfloor$, $n_2 = \lceil n / 2 \rceil$.
- Induction step: assume true for $1, 2, \dots, n-1$.

$$\begin{aligned} T(n) &\leq T(n_1) + T(n_2) + n \\ &\leq n_1 \lceil \log_2 n_1 \rceil + n_2 \lceil \log_2 n_2 \rceil + n \\ &\leq n_1 \lceil \log_2 n_2 \rceil + n_2 \lceil \log_2 n_2 \rceil + n \\ &= n \lceil \log_2 n_2 \rceil + n \\ &\leq n(\lceil \log_2 n \rceil - 1) + n \\ &= n \lceil \log_2 n \rceil \end{aligned}$$

$$\begin{aligned} n_2 &= \lceil n / 2 \rceil \\ &\leq \lceil 2^{\lceil \log_2 n \rceil} / 2 \rceil \\ &= 2^{\lceil \log_2 n \rceil} / 2 \\ &\Rightarrow \log_2 n_2 \leq \lceil \log_2 n \rceil - 1 \end{aligned}$$

5.4 Closest Pair of Points

Closest Pair of Points

Closest pair. Given n points in the plane, find a pair with smallest Euclidean distance between them.

Fundamental geometric primitive.

- Graphics, computer vision, geographic information systems, molecular modeling, air traffic control.
- Fast closest pair inspired fast algorithms for these problems: nearest neighbor, Euclidean MST, Voronoi.

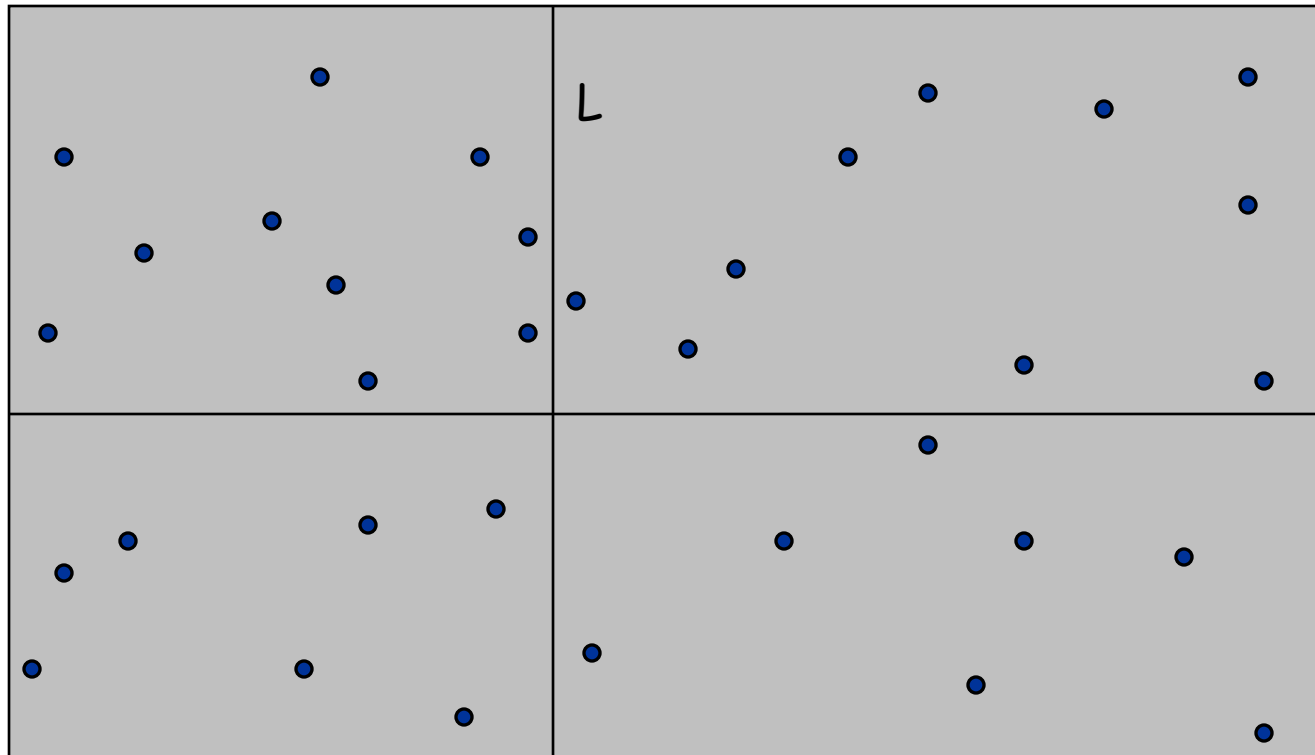
Brute force. Check all pairs of points p and q with $\Theta(n^2)$ comparisons.

Assumption. No two points have same x coordinate.

↑
to make presentation cleaner

Closest Pair of Points: First Attempt

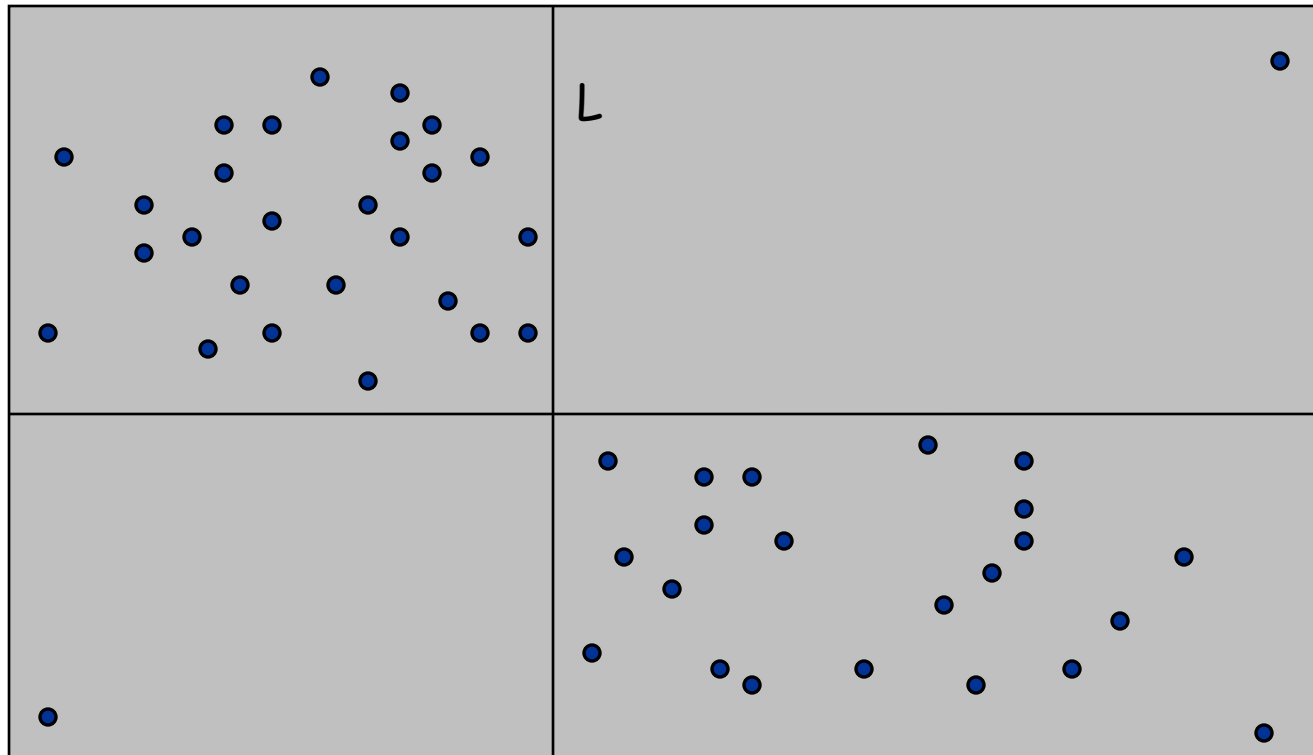
Divide. Sub-divide region into 4 quadrants.



Closest Pair of Points: First Attempt

Divide. Sub-divide region into 4 quadrants.

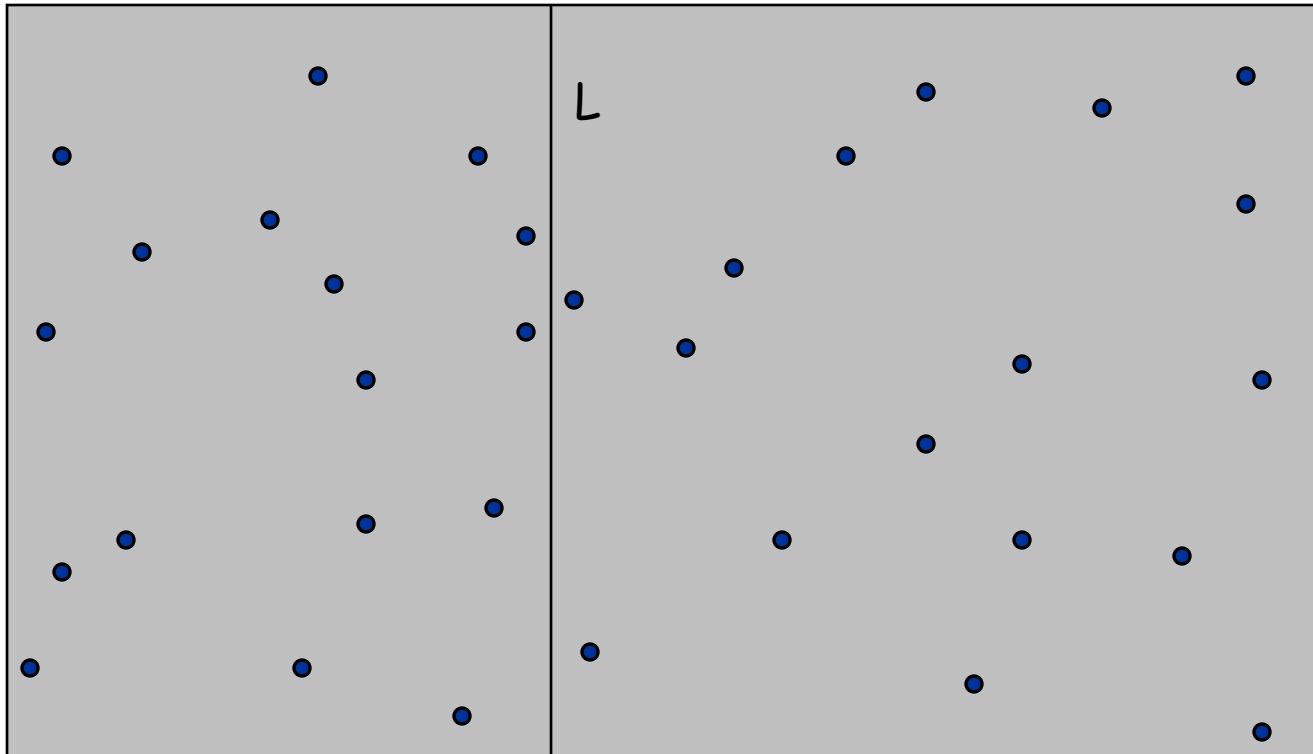
Obstacle. Impossible to ensure $n/4$ points in each piece.



Closest Pair of Points

Algorithm.

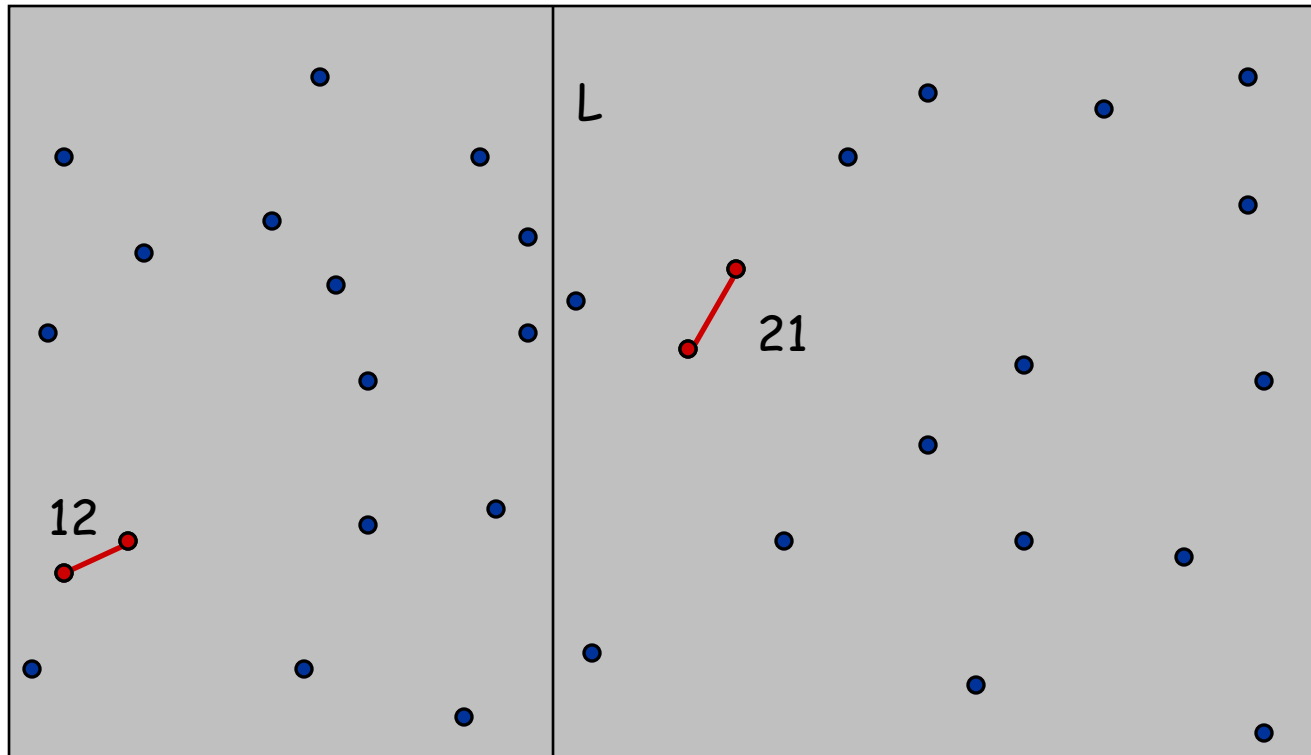
- **Divide:** draw vertical line L so that roughly $\frac{1}{2}n$ points on each side.



Closest Pair of Points

Algorithm.

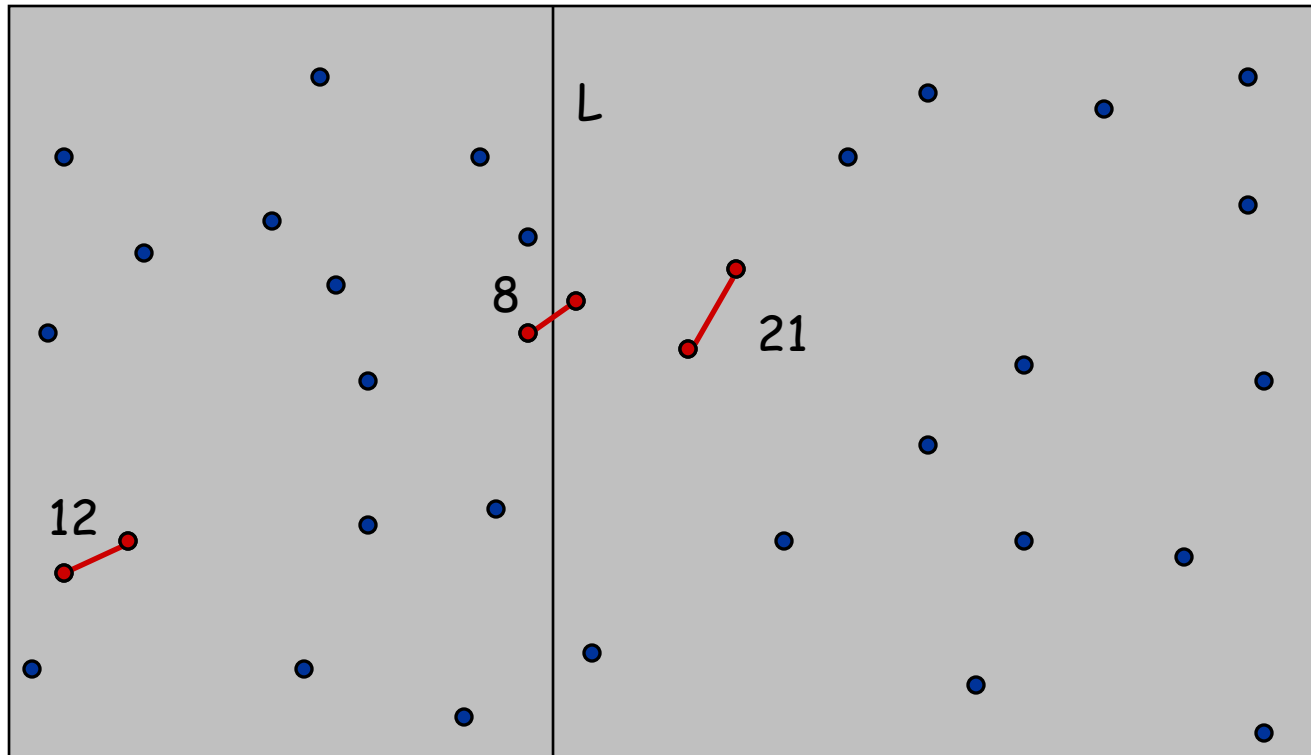
- Divide: draw vertical line L so that roughly $\frac{1}{2}n$ points on each side.
- **Conquer:** find closest pair in each side recursively.



Closest Pair of Points

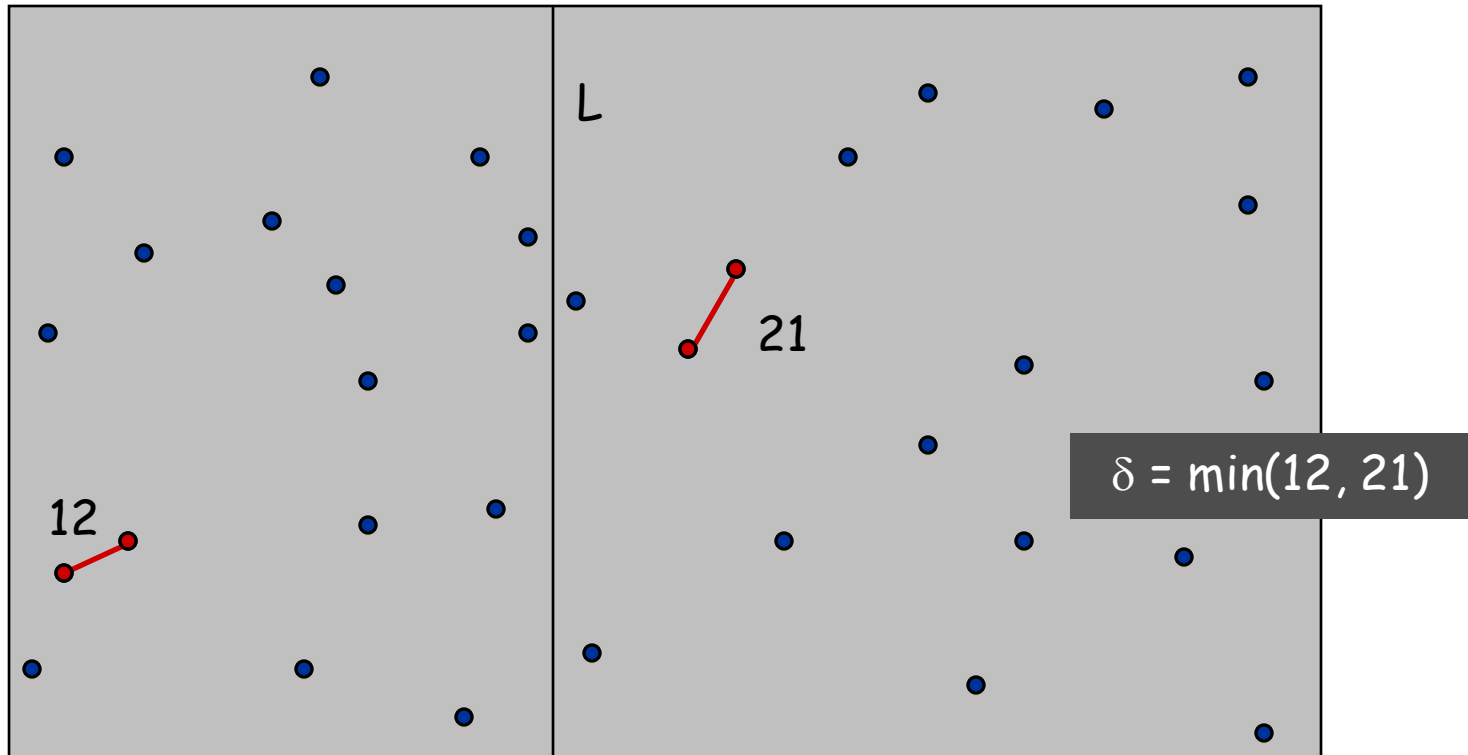
Algorithm.

- Divide: draw vertical line L so that roughly $\frac{1}{2}n$ points on each side.
- Conquer: find closest pair in each side recursively.
- **Combine:** find closest pair with one point in each side. ← seems like $\Theta(n^2)$
- Return best of 3 solutions.



Closest Pair of Points

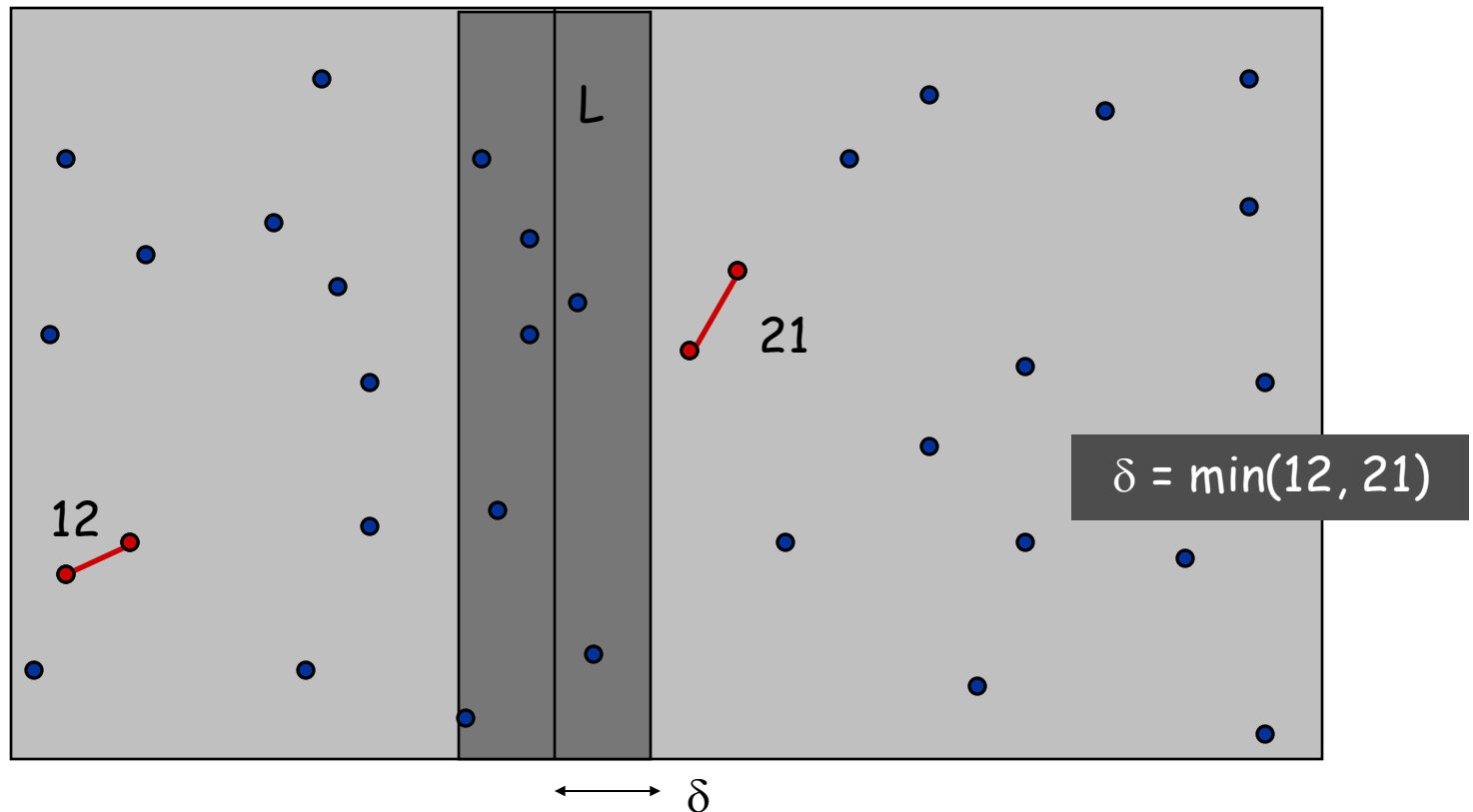
Find closest pair with one point in each side, **assuming that distance $< \delta$** .



Closest Pair of Points

Find closest pair with one point in each side, **assuming that distance $< \delta$** .

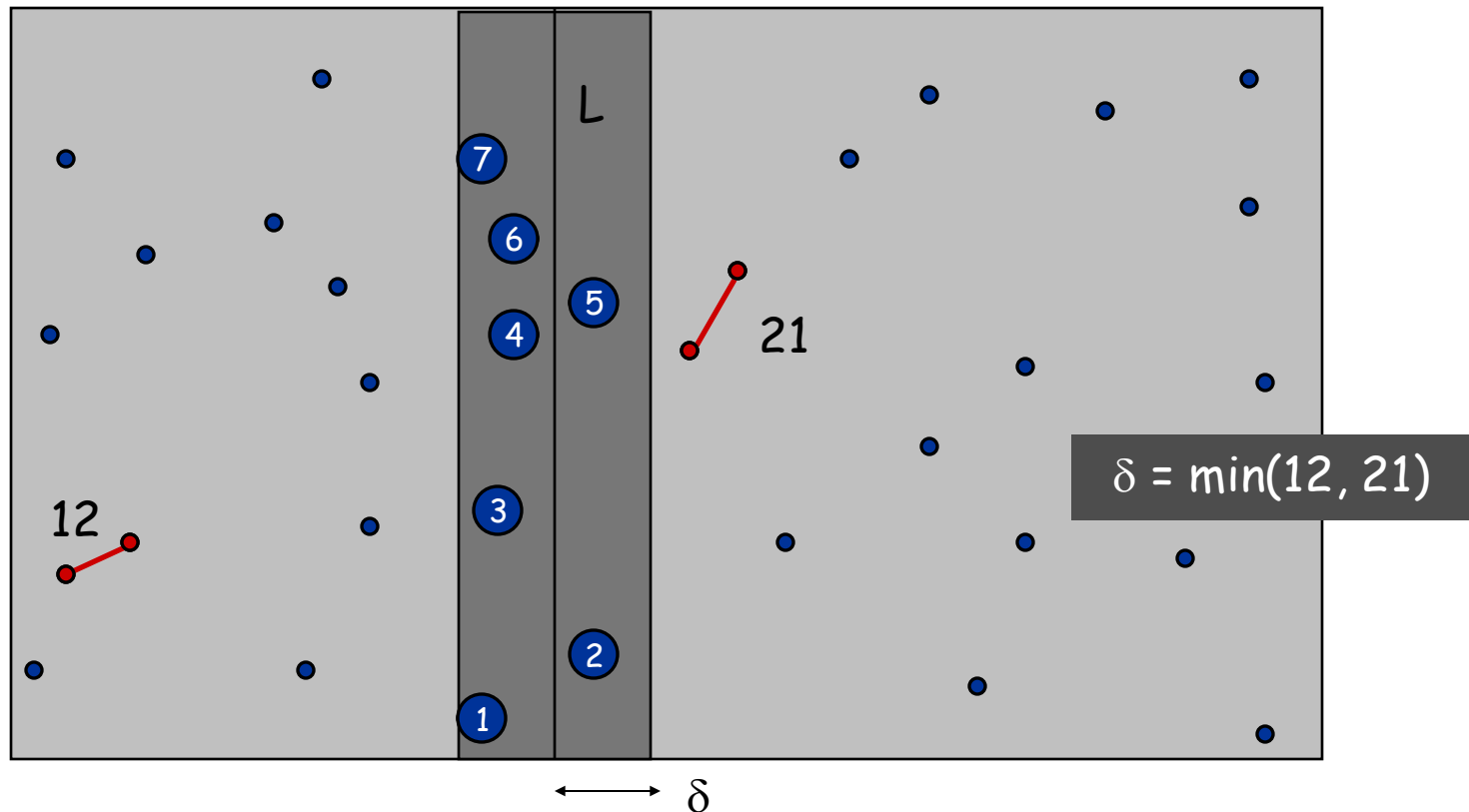
- Observation: only need to consider points within δ of line L .



Closest Pair of Points

Find closest pair with one point in each side, **assuming that distance $< \delta$** .

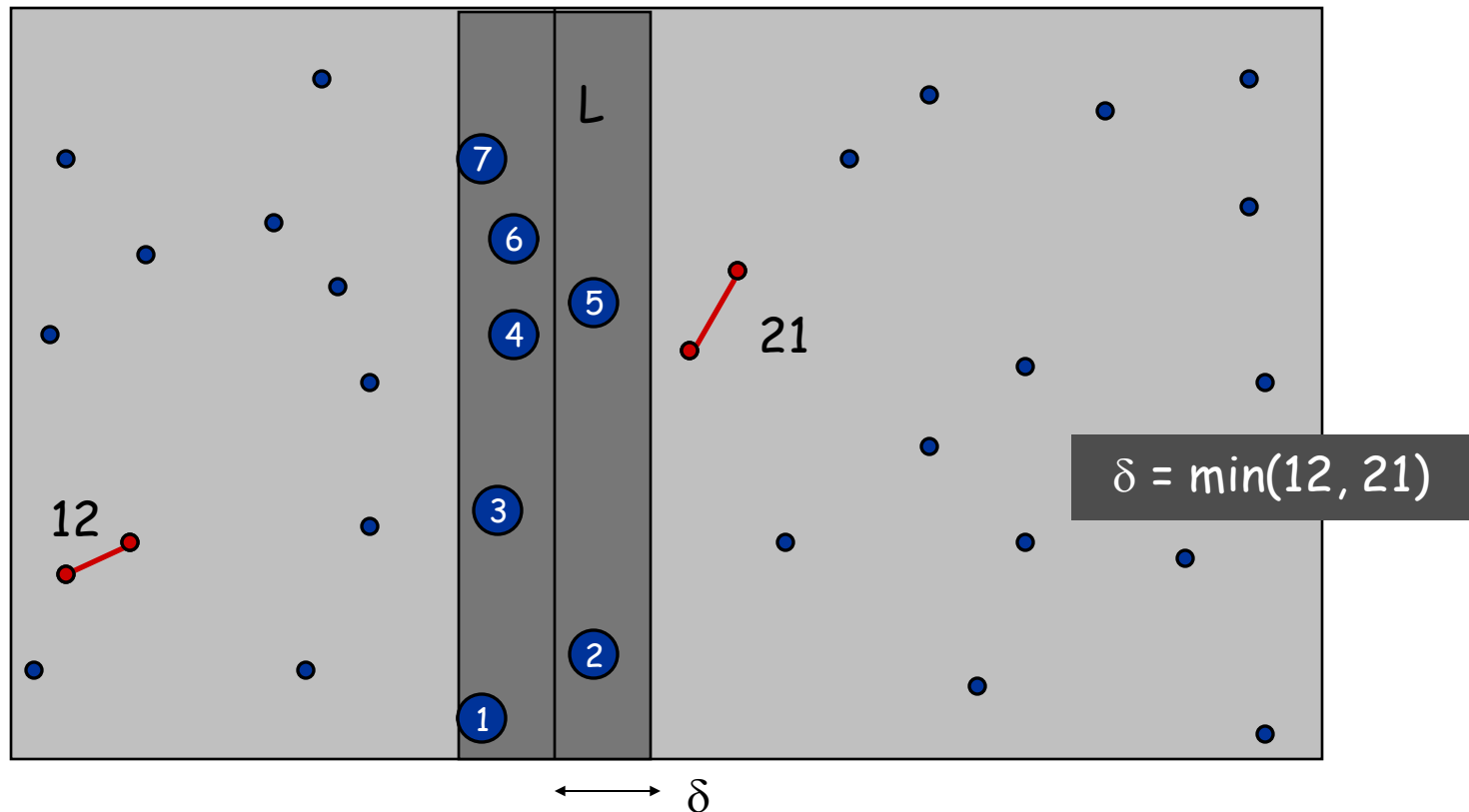
- Observation: only need to consider points within δ of line L .
- Sort points in 2δ -strip by their y coordinate.



Closest Pair of Points

Find closest pair with one point in each side, **assuming that distance $< \delta$** .

- Observation: only need to consider points within δ of line L .
- Sort points in 2δ -strip by their y coordinate.
- Only check distances of those within 11 positions in sorted list!



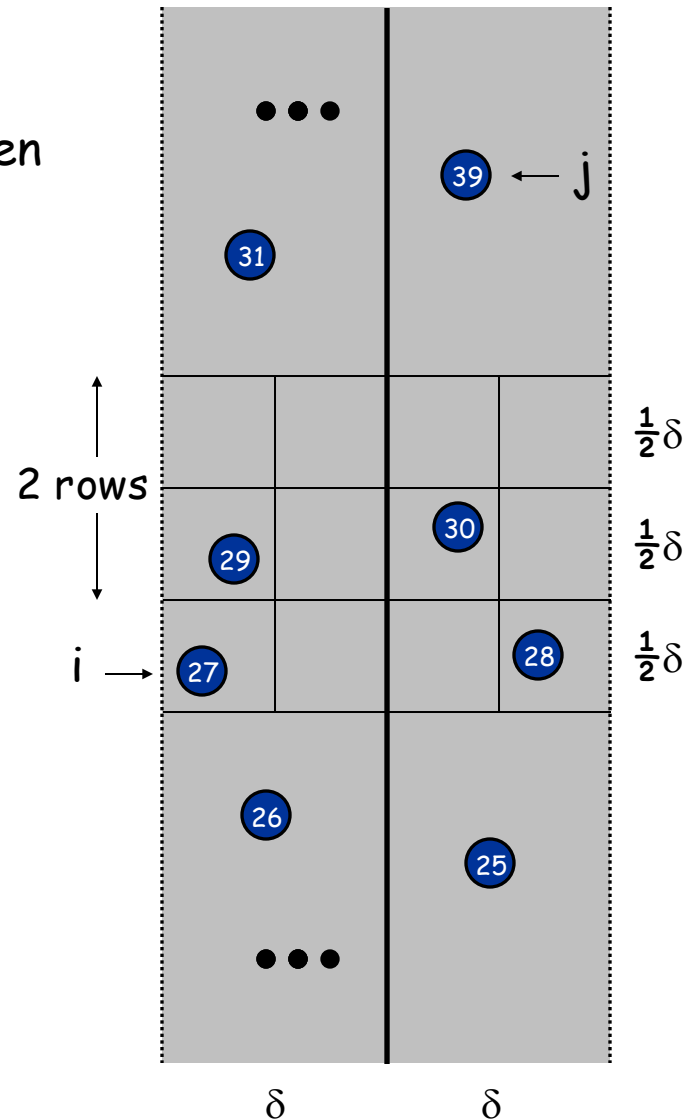
Closest Pair of Points

Def. Let s_i be the point in the 2δ -strip, with the i^{th} smallest y-coordinate.

Claim. If $|i - j| \geq 12$, then the distance between s_i and s_j is at least δ .

Pf.

- No two points lie in same $\frac{1}{2}\delta$ -by- $\frac{1}{2}\delta$ box.
- Two points at least 2 rows apart have distance $\geq 2(\frac{1}{2}\delta)$. ▫



Fact. Still true if replace 12 with 8. How?

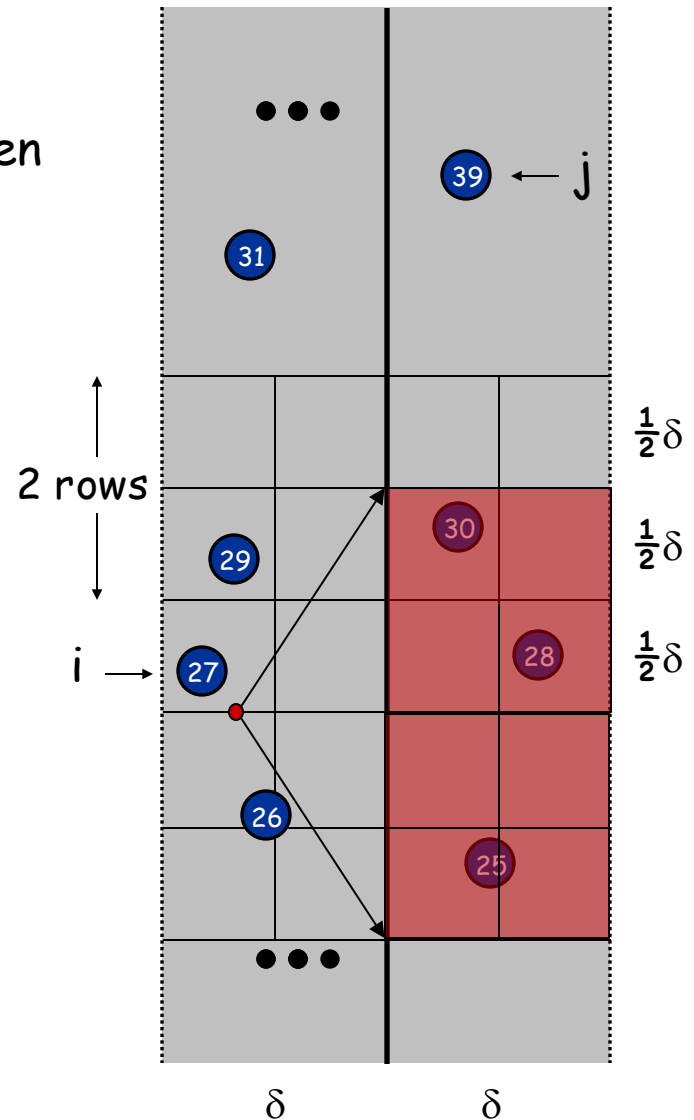
Closest Pair of Points

Def. Let s_i be the point in the 2δ -strip, with the i^{th} smallest y-coordinate.

Claim. If $|i - j| \geq 12$, then the distance between s_i and s_j is at least δ .

Pf.

- No two points lie in same $\frac{1}{2}\delta$ -by- $\frac{1}{2}\delta$ box.
- Two points at least 2 rows apart have distance $\geq 2(\frac{1}{2}\delta)$. ▀



Fact. Still true if replace 12 with 8. How?

Closest Pair of Points

Find closest pair with one point in each side, assuming that distance $< \delta$.

- Linear time algorithm!

Without this assumption?

- Run the same algorithm
- If assumption true, we will find the right closest pair with one point in each side
- If false, the algorithm will find a pair with distance $\geq \delta$, and then the combine step will correctly return δ as distance of closest pair

Closest Pair Algorithm

```
Closest-Pair( $p_1, \dots, p_n$ ) {  
    Compute separation line  $L$  such that half the points  
        are on one side and half on the other side.  $O(n \log n)$   
  
     $\delta_1$  = Closest-Pair(left half)  $2T(n / 2)$   
     $\delta_2$  = Closest-Pair(right half)  
     $\delta$  =  $\min(\delta_1, \delta_2)$   
  
    Delete all points further than  $\delta$  from separation line  $L$   $O(n)$   
  
    Sort remaining points by  $y$ -coordinate.  $O(n \log n)$   
  
    Scan points in  $y$ -order and compare distance between  
        each point and next 11 neighbors. If any of these  
        distances is less than  $\delta$ , update  $\delta$ .  $O(n)$   
  
    return  $\delta$ .  
}
```

Closest Pair of Points: Analysis

Running time.

$$T(n) \leq 2T(n/2) + O(n \log n) \Rightarrow T(n) = O(n \log^2 n)$$

Q. Can we achieve $O(n \log n)$?

Sorting R points by y coordinate

At the beginning of the algorithm, also sort all the points vertically. Store them in a separate array V .

- Takes $O(n \log n)$ time.

Points in H and V have pointers to each other.

- I.e. given p in H , its pointer gives p 's index in V . Similarly given p in V , we can get p 's index in H .

When picking out points left (or right) of dividing line using H , mark them in V by following the pointers.

Next, iterate through V (in vertical order) and pick out marked points.

- These points are again sorted vertically.
- Takes $O(n)$ time.

Add this $O(n \log n)$ preprocessing time to algorithm's running time. Algorithm still $O(n \log n)$.

Closest Pair of Points: Analysis

Running time.

$$T(n) \leq 2T(n/2) + O(n \log n) \Rightarrow T(n) = O(n \log^2 n)$$

Q. Can we achieve $O(n \log n)$?

A. Yes. Don't sort points from scratch each time.

- Sort all the points twice before recursive call, once by x coordinate and once by y coordinate
- Reuse the sorted sequences when needed (linear time)

$$T(n) \leq 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$$

Selection

Given list A of n numbers, and $i \leq n$, $\text{Select}(A, i)$ finds i 'th smallest number in A .

- **Ex** $A = [3, 1, 6, 7, 2]$, $\text{Select}(A, 4) = 6$.
- Assume numbers all distinct, for simplicity.

Select generalizes median.

- Median of A is $\text{Select}(A, n/2)$.

We can solve select by first sorting the numbers in $O(n \log n)$ time, then choosing the i 'th largest one.

We show how to solve Select , and hence median, in $O(n)$ time.

Select algorithm overview

Break A into many small groups.

Find the median of each small group directly.

Recursively find the median of this set of medians (using Select).

Use the median-of-medians to partition A .

Keep looking for target in one of the two partitions.

Select algorithm 1

Say A has n items, and we want i 'th largest.

- Assume for simplicity n divides 5.

Divide A into $n/5$ groups of 5 elements each.

- First 5 elements of A in first group, next 5 in second group, etc.

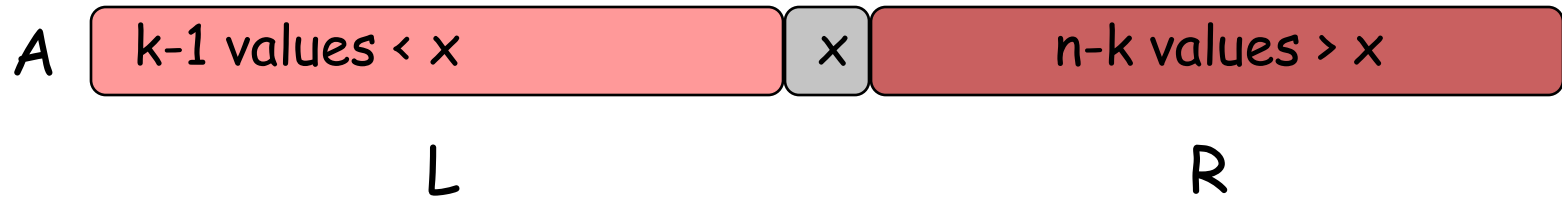
For each group of 5 elements, find its median.

- E.g. sort the 5 numbers, take 3rd value.
- Let B be the set of $n/5$ medians.

Recursively find the median in B .

- I.e. do $\text{Select}(B, n/10)$, finding median in B .
- Say the median is x .

Select algorithm 2



Partition A using x as pivot.

- Move all values $< x$ to the left of x , all other values to the right.
- If A has n values, this takes $O(n)$ time.

Say there are k values $\leq x$ in A , and $n - k$ values $> x$.

If $i = k$, return x .

- We want the i 'th largest item in A , which is x .

If $i < k$, return $\text{Select}(L, i)$.

- **Ex** If $i = 5$ and $k = 10$, then fifth largest item in A is fifth largest item in L .

Else $i > k$, return $\text{Select}(R, i - k)$.

- **Ex** if $i = 15$ and $k = 10$, then fifteenth largest item in A is fifth largest item in R .

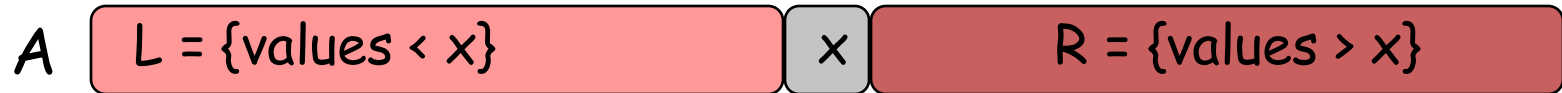
Select analysis 1

Let $S(n)$ be an upper bound on select's running time given a list of size n .

$$S(n) = S(n/5) + S(u) + O(n)$$

- u is the size of whichever partition we recurse on.
- $O(n)$ term comes from two parts.
 - First, for each of the $n/5$ groups of 5 numbers, find their median in constant time $\Rightarrow O(n)$ time overall.
 - Next, partition array in $O(n)$ time.
- $S(n/5)$ to find the median of the $n/5$ group medians.
- The key to ensuring select runs fast is ensuring u is small.
- We will show $u \leq 7n/10$.

Select analysis 2



Let x be the median of medians.

We show that $|L| \geq 3n/10$, and $|R| \geq 3n/10$.

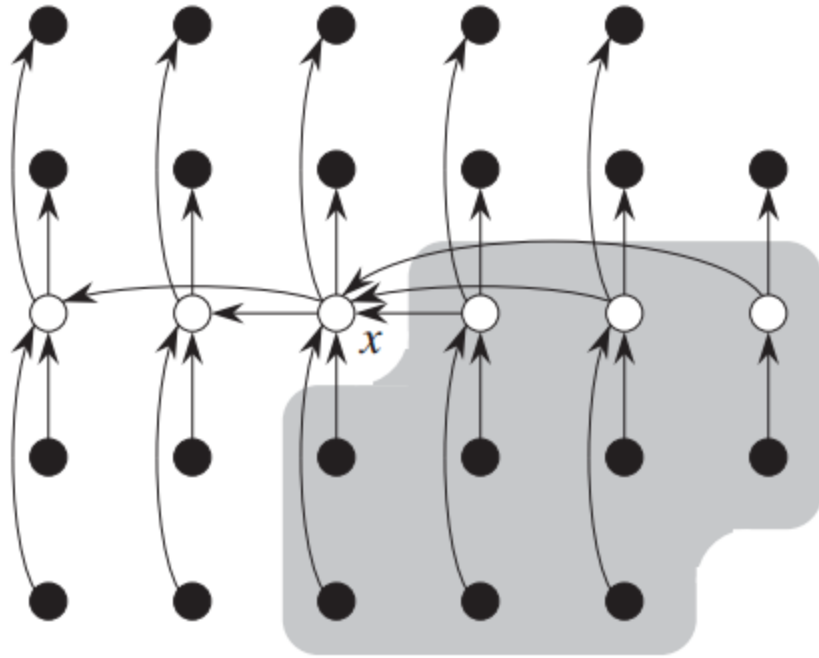
- I.e. there are at least $3n/10$ values less than x , and $3n/10$ values larger than x .

Then, also get $|L| \leq 7n/10$.

- $|L| = n - 1 - |R|$.
- Also, $|R| \leq 7n/10$.

Thus, $S(n) \leq S(n/5) + S(7n/10) + O(n)$.

Select analysis 3



- Groups of 5 elements are shown in columns.
- Medians are shown in white.
- Medians less than x are shown to the right.

There are $n/5 * 1/2 = n/10$ medians less than x .

For each such median, there are two more values from the group less than the median.

- There are 3 values from each group $< x$.

So there are at least $3n/10$ values $< x$.

I.e. $|L| \geq 3n/10$.

Similarly, $|R| \geq 3n/10$.

Select analysis 4

We have $S(n) \leq S(n/5) + S(7n/10) + O(n)$.

- Since we want to upper bound $S(n)$, we let $S(n) = S(n/5) + S(7n/10) + O(n)$.
- For sufficiently large n , the $O(n)$ term is bounded by bn , for some constant b .

We guess $S(n) = O(n)$, and show this satisfies equation above, i.e. the guess is valid.

Since $S(n) = O(n)$, $S(n) \leq cn$ for sufficiently large n , for some constant c .

Select analysis 5

From $S(n) = S(n/5) + S(7n/10) + bn$ and $S(n) = cn$, we get

$$cn = c(n/5) + c * 7n/10 + bn.$$

So $cn = 9cn/10 + bn$, and $c = 10b$.

Thus, $S(n) \leq 10bn = O(n)$ for sufficiently large n , for some constant b .